

内存屏障：软件黑客的硬件视角

保罗·E·麦肯尼 IBM Linux
技术中心 贝夫顿实验室
paulmck@us.ibm.com

2009年4月5日

是什么让CPU设计者做出这样的决定，迫使他们将内存屏障强加给无辜且不知情的SMP软件设计者？

简而言之，由于重新排序内存引用可以显著提升性能，因此在诸如正确运行依赖于有序内存引用的同步原语等场景中，需要使用内存屏障来强制执行顺序。

要更详细地回答这个问题，需要深入了解CPU缓存的工作原理，特别是使缓存真正高效运行所需的条件。以下各节：

呈现缓存的结构，

2. 描述缓存一致性协议如何确保各个CPU对内存中每个位置的值达成一致，并最终实现这一点。
3. 描述存储缓冲区和无效队列如何帮助缓存及缓存一致性协议实现高性能。

内存屏障是一种必要的机制，虽然它带来了复杂性，但却是实现良好性能和可扩展性的前提。这种复杂性源于CPU的运行速度远远超过其间的互连以及它们试图访问的内存的速度。

1 Cache Structure

现代CPU的运行速度远超现代内存系统。一个2006年的CPU可能具备每秒执行数十亿次操作的能力，而内存系统的性能则远远落后。

在一个纳秒内可以执行十条指令，但需要数十纳秒才能从主内存中获取一个数据项。这种速度上的差异——相差两个数量级以上——导致了现代CPU上出现的多兆字节缓存。这些缓存与CPU相关联，如图1所示，并且通常可以在几个时钟周期内访问到。

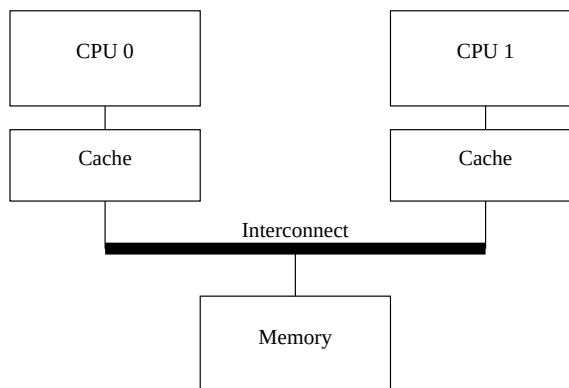


图1：现代计算机系统缓存结构

数据在CPU缓存和内存之间以固定长度的块（称为“缓存行”）进行传输，这些块通常为2的幂大小，范围从16字节到256字节。当某个数据项首次被访问时

¹It is standard practice to use multiple levels of cache, with a small level-one cache close to the CPU with single-cycle access time, and a larger level-two cache with a longer access time, perhaps roughly ten clock cycles. Higher-performance CPUs often have three or even four levels of cache.

对于给定的CPU而言，该数据将不会存在于其缓存中，这意味着发生了“缓存未命中”（更具体地说是“启动”或“预热”阶段的缓存未命中）。缓存未命中意味着CPU必须等待数百个时钟周期，直到从内存中获取该项数据。然而，该项数据会被加载到该CPU的缓存中，因此后续访问将直接在缓存中找到该项数据，并以全速运行。

经过一段时间，CPU的缓存将被填满，随后的缺失（缺页异常）很可能需要从缓存中移除一个项目以腾出空间给新获取的项目。这种缓存缺失被称为“容量缺失”，因为它是由缓存有限的容量引起的。然而，大多数缓存即使在未完全填满的情况下也可以被迫移除旧项来为新项腾出空间。这是由于大容量缓存通常作为硬件哈希表实现，其中包含固定大小的哈希桶（或称为“集合”，如CPU设计者所称），并且不使用链接机制，如图2所示。

此缓存包含十六个“组”和两个“路”，总共为32条“缓存行”，每个条目包含一个单独的256字节“缓存行”，即内存中的一个256字节对齐的块。这种缓存行大小略大一些，但使十六进制运算变得简单得多。在硬件术语中，这是一个两路组相联缓存，并且类似于具有十六个桶的软件哈希表，其中每个桶的哈希链最多包含两个元素。该缓存的大小（本例中有32条缓存行）和关联性（本例中为两路）统称为缓存的“几何”。由于此缓存是在硬件中实现的，因此哈希函数非常简单：从内存地址中提取四个比特。

如图2所示，每个方框对应一个缓存条目，可以包含一条256字节的缓存行。然而，缓存条目也可以为空，如图中空方框所示。其余的方框则标记有其所含缓存行的内存地址。由于缓存行必须是256字节对齐的，因此每个地址的低8位为0，并且硬件哈希函数的选择意味着接下来的4位与哈希线号匹配。

该图所示的情况可能会出现在程序代码位于地址的情况下。

	Way 0	Way 1
0x0	0x12345000	
0x1	0x12345100	
0x2	0x12345200	
0x3	0x12345300	
0x4	0x12345400	
0x5	0x12345500	
0x6	0x12345600	
0x7	0x12345700	
0x8	0x12345800	
0x9	0x12345900	
0xA	0x12345A00	
0xB	0x12345B00	
0xC	0x12345C00	
0xD	0x12345D00	
0xE	0x12345E00	0x43210E00
0xF		

图2：CPU缓存结构

0x43210E00 至 0x43210EFF，该程序依次访问了从 0x12345000 到 0x12345EFF 的数据。假设程序现在要访问位置 0x12345F00。此位置哈希到缓存行 0xF，并且该行的两个路均为空，因此可以容纳对应的 256 字节缓存行。如果程序访问位置 0x1233000（该位置哈希到缓存行 0x0），则可以在路 1 中容纳对应的 256 字节缓存行。然而，若程序访问位置 0x1233E00（该位置哈希到缓存行 0xE），必须将现有的一条缓存行从缓存中移除以腾出空间给新的缓存行。如果后来再次访问被移除的这条缓存行，则会导致缓存未命中。这种类型的缓存未命中被称为“关联性未命中”。

到目前为止，我们只考虑了CPU读取数据项的情况。当CPU执行写操作时会发生什么？由于所有CPU必须对给定数据项的值达成一致，因此在某个CPU向该数据项写入之前，它必须首先导致其他CPU缓存中的该数据项被移除或“失效”。一旦完成此失效过程，该CPU可以安全地修改数据项。如果该数据项存在于当前CPU的缓存中但为只读，则这一过程称为“写缺失”。在某个CPU完成了将给定数据项从其他CPU缓存中失效后，该CPU可以重复上述步骤。

正确写入（并读取）该数据项。

后来，如果其他某个CPU尝试访问该数据项时，将会发生缓存未命中，这次是因为第一个CPU为了写入数据而使该项失效。这种类型的缓存未命中被称为“通信未命中”，因为通常这是由于多个CPU使用数据项进行通信所导致的（例如，一个锁就是一个用于在采用互斥算法的CPU之间进行通信的数据项）。

显然，必须采取周密的措施确保所有CPU对数据保持一致的看法。在不断获取、失效和写入的过程中，很容易想象数据可能会丢失，或者（也许更糟糕的是）不同的CPU在其各自的缓存中为同一数据项持有冲突的值。这些问题通过“缓存一致性协议”来预防，这些协议将在下一节中描述。

2 Cache-Coherence Protocols

缓存一致性协议管理缓存行的状态，以防止数据不一致或丢失。这些协议可能非常复杂，包含数十种状态，但对于我们而言，只需关注四态的MESI缓存一致性协议即可。

2.1 MESI States

MESI 分别代表“修改”、“独占”、“共享”和“无效”，这是使用该协议的缓存行可以处于的四种状态。采用此协议的缓存因此会在每条缓存行上除了物理地址和数据之外，还维护一个两位的状态标记（标签）。

处于“修改”状态的缓存行最近曾被相应的CPU进行了内存写操作，且对应的内存数据保证不会出现在其他任何CPU的缓存中。因此，可以将处于“修改”状态的缓存行说成是“被相应CPU所独占”的。

²See Culler et al. [3] pages 670 and 671 for the nine-state and 26-state diagrams for SGI Origin2000 and Sequent (now IBM) NUMA-Q, respectively. Both diagrams are significantly simpler than real life.

由于这个缓存中持有数据的唯一最新副本，因此该缓存最终负责将数据写回内存或将数据传递给其他缓存，并且必须在重用此缓存行来存储其他数据之前完成上述操作。

“独占”状态与“修改”状态非常相似，唯一的区别在于缓存行尚未被相应的CPU修改过，这意味着内存中该缓存行数据的副本是最新的。然而，由于CPU可以在任何时候存储到这条缓存行而无需与其他CPU进行通信，“独占”状态下的缓存行仍然可以被认为是相应CPU拥有的。不过，因为内存中的值是最新的，所以这个缓存可以丢弃这些数据而不必将其写回内存或将数据传递给其他CPU。

处于“共享”状态的一行可能还被至少一个其他CPU的缓存复制，因此该CPU在对该行进行写操作之前必须先与其他CPU进行协商。与“独占”状态类似，由于内存中的相应值是最新的，此缓存可以丢弃这些数据而无需将其回写到内存或将数据传递给其他CPU。

处于“无效”状态的缓存行是空的，换句话说，它不包含任何数据。当新数据进入缓存时，如果可能的话，会将其放置到一个处于“无效”状态的缓存行中。这种做法被优先采用是因为替换其他任何状态下的缓存行可能会导致将来引用该缓存行时发生代价高昂的缓存未命中。

由于所有CPU都必须维护缓存行中数据的一致视图，因此缓存一致性协议提供了协调缓存行在系统中移动的消息。

2.2 MESI Protocol Messages

许多前一节中描述的转换需要在CPU之间进行通信。如果这些CPU位于同一共享总线上，则以下消息就足够了：

Read: “读取”消息包含了要读取的缓存行的物理地址。

Read Response: “读响应”消息包含了一条早期“读”消息请求的数据。这条“读响应”消息可能是由内存提供，也可能是由其他缓存之一提供的。例如，如果某个缓存在“修改”状态中有所需数据，则该缓存必须提供“读响应”消息。

Invalidate: “无效”消息包含要被标记为无效的缓存行的物理地址。所有其他缓存必须从其缓存中移除相应数据并作出响应。

Invalidate Acknowledge: 接收“无效”消息的CPU必须在从其缓存中移除指定数据后，回复一个“无效确认”消息。

Read Invalidate: “读无效”消息包含了要读取的缓存行的物理地址，同时指示其他缓存移除数据。因此，它结合了“读取”和“无效”的功能，正如其名称所示。“读无效”消息需要一个“读响应”以及一系列“无效确认”消息作为回复。

Writeback: “写回”消息包含要写回到内存中的地址和数据（并且可能在途中被其他CPU的缓存监听到）。此消息允许缓存在需要时将处于“修改”状态的行逐出，以腾出空间供其他数据使用。

有趣的是，一个共享内存的对称多处理器系统实际上是一种基于消息传递的计算机。这意味着使用分布式共享内存的SMP机器集群在系统的两个不同层次架构上通过消息传递来实现共享内存。

Quick Quiz 1: 两个CPU同时尝试无效化同一缓存行会发生什么？

Quick Quiz 2: 当一个“无效”消息出现在大型多处理器系统中时，每个CPU都必须给出一个“无效确认”响应。这样的“无效确认”响应风暴会不会完全饱和系统总线？

Quick Quiz 3: 如果SMP机器实际上已经在使用消息传递机制，那么为何还要使用SMP呢？

2.3 MESI State Diagram

给定的缓存行的状态随着协议消息的发送和接收而发生变化，如图3所示。

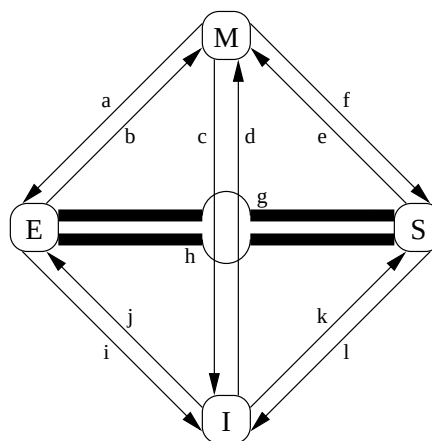


图3: MESI缓存一致性状态图

该图中的转换弧如下：

Transition (a): 缓存行被写回内存，但CPU仍然将其保留在缓存中，并保留进一步修改的权利。这一过渡需要发送一个“写回”消息。

Transition (b): CPU将其已获得独占访问权的缓存行进行写入。此转换无需发送或接收任何消息。

Transition (c): CPU收到一个针对已被修改的缓存行的“读无效”消息。CPU必须使其本地副本失效，然后回应一条包含数据并发送给请求方的“读响应”消息以及一条表示不再持有该缓存行本地副本的“无效确认”消息。

Transition (d): CPU对缓存中不存在的数据项执行原子读取-修改-写入操作。它发送“读无效”请求，并通过“读响应”接收数据。一旦收到完整的“无效确认”响应集，CPU即可完成该转换过程。

Transition (e): CPU 对之前只读缓存中的一个数据项执行原子的读取-修改-写入操作。它必须传输“无效”消息，并且在完成转换前必须等待完整的一组“无效确认”响应。

Transition (f): 其他CPU读取了缓存行，该缓存行来源于此CPU的缓存，其中保留了一份只读副本。这一状态转换由接收“读”消息触发，随后此CPU用包含请求数据的“读响应”消息进行回复。

Transition (g): 其他CPU读取了该缓存行中的一个数据项，这个数据要么来自此CPU的缓存，要么来自内存。无论哪种情况，此CPU都会保留一份只读副本。这一转换由接收“读”消息触发，并且此CPU会用包含请求数据的“读响应”消息进行回应。

Transition (h): 该CPU意识到它即将需要向此缓存行中的某个数据项写入数据，因此发送了一个“无效”消息。该CPU必须等到收到所有其他CPU的“无效确认”响应后才能完成这一转换过程。或者，所有其他CPU通过发送“回写”消息将此缓存行从它们的缓存中剔除（可能是为了给其他缓存行腾出空间），从而使该CPU成为最后一个保留此缓存行的CPU。

Transition (i): 另一颗CPU对仅存于本CPU缓存中的某缓存行内的数据项执行了原子读改写操作，因此本CPU将其从缓存中失效。这一转换由接收“读无效”消息触发，随后本CPU响应为

both 读响应 和 无效确认 消息。

Transition (j): 此CPU将数据项存储到缓存中未包含的缓存行，并因此传输一个“读无效”消息。该CPU无法完成转换，直到收到“读响应”和一整套“无效确认”消息。假设在实际写入完成后，该缓存行会通过转换(b)立即转变为“已修改”状态。

Transition (k): 该CPU加载了一个不在其缓存中的缓存行数据项。CPU发送一个“读取”消息，并在收到相应的“读取响应”后完成过渡。

Transition (l): 另一些CPU对该缓存行中的数据项进行了写操作，但由于该缓存行被其他CPU的缓存（包括当前CPU的缓存）以只读状态持有，因此它保持了只读状态。这一状态转换是由接收“无效”消息触发的，并且此CPU会回应一个“无效确认”消息。

Quick Quiz 4: 硬件是如何处理上述延迟过渡的？

2.4 MESI Protocol Example

现在我们从缓存行的角度来看这个问题，初始时该缓存行的数据位于地址为0的内存中，在一个四CPU系统的各个单线直接映射缓存之间流转。表1展示了这一数据流情况，第一列显示操作序列，第二列显示执行操作的CPU，第三列显示正在进行的操作，接下来的四列表示每个CPU缓存行的状态（包括内存地址和MESI状态），最后两列表示相应内存内容是否是最新的（“V”表示最新，“I”表示不最新）。

初始时，数据将要驻留的CPU缓存行处于“无效”状态，而内存中的数据是有效的。当CPU 0加载该地址的数据时

地址为0的数据进入CPU 0缓存的“共享”状态，并且在内存中仍然有效。接着，CPU 3也加载了地址为0的数据，因此该数据同时处于两个CPU缓存的“共享”状态，在内存中依然有效。随后，CPU 3加载了另一个缓存行（地址为8），这导致其通过写回操作将地址为0的数据从缓存中移除，并用地址为8的数据替换它。此时，CPU 2尝试从地址0读取数据，但意识到很快需要对该地址进行存储操作，因此使用“读无效”消息来获取独占副本并使该数据在CPU 3的缓存中失效（尽管内存中的副本仍然有效）。接下来，CPU 2执行了预期的存储操作，将状态更改为“修改”。此时，内存中的数据副本已经过时。接着，CPU 1进行原子增量操作，使用“读无效”消息从CPU 2的缓存中窥探数据并使其失效，使得CPU 1缓存中的数据副本处于“修改”状态（而内存中的副本仍然过时）。最后，CPU 1读取地址为8的缓存行，并通过“写回”消息将地址0的数据推回到内存。

注意，我们在某些CPU缓存中结束了数据的处理。

Quick Quiz 5: 什么样的操作序列可以将所有CPU缓存都重新置为“无效”状态？

3 Stores Result in Unnecessary Stalls

尽管图1所示的缓存结构对于给定CPU对特定数据项进行重复读写时性能良好，但对于首次向给定缓存行写入的情况，其性能却非常差。为了说明这一点，请参见图4，该图展示了CPU 0向位于CPU 1缓存中的某缓存行进行写操作的时间线。由于CPU 0必须等待缓存行到达后才能对其进行写入，因此CPU 0需要长时间停滞。

³The time required to transfer a cache line from one CPU's cache to another's is typically a few orders of magnitude more than that required to execute a simple register-to-register instruction.

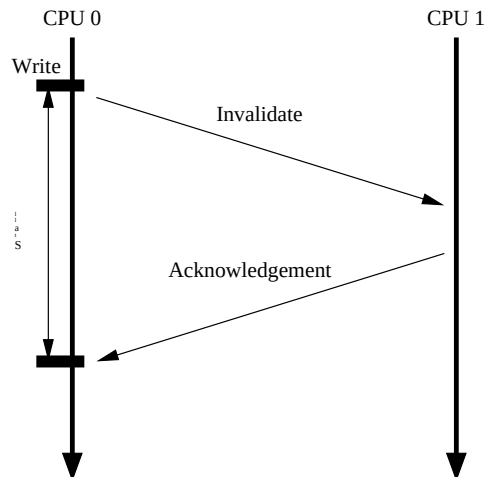


图4：写操作不必要的停顿

但并没有真正的原因非要让CPU 0长时间停顿——毕竟，无论CPU 1发送的缓存行中包含什么数据，CPU 0都会无条件地将其覆盖。

3.1 Store Buffers

一种防止写操作不必要的停滞的方法是在每个CPU与其缓存之间添加“存储缓冲区”，如图5所示。通过增加这些存储缓冲区，CPU 0可以简单地将其写入记录在存储缓冲区内并继续执行。当缓存行最终从CPU 1传输到CPU 0时，数据将从存储缓冲区移动到缓存行。

然而，接下来的两个部分将讨论必须解决的一些复杂问题。

3.2 Store Forwarding

为了看到第一个复杂情况，即自一致性被违反的情况，请考虑以下代码：变量“a”和“b”初始值均为零，并且包含变量“a”的缓存行最初由CPU 1拥有，而包含变量“b”的缓存行最初由CPU 0拥有。

Sequence #	CPU #	Operation	CPU Cache				Memory	
			0	1	2	3	0	8
0		Initial State	-/I	-/I	-/I	-/I	V	V
1	0	Load	0/S	-/I	-/I	-/I	V	V
2	3	Load	0/S	-/I	-/I	0/S	V	V
3	0	Writeback	8/S	-/I	-/I	0/S	V	V
4	2	RMW	8/S	-/I	0/E	-/I	V	V
5	2	Store	8/S	-/I	0/M	-/I	1	V
6	1	Atomic Inc	8/S	0/M	-/I	-/I	1	V
7	1	Writeback	8/S	8/S	-/I	-/I	V	V

表1：缓存一致性示例

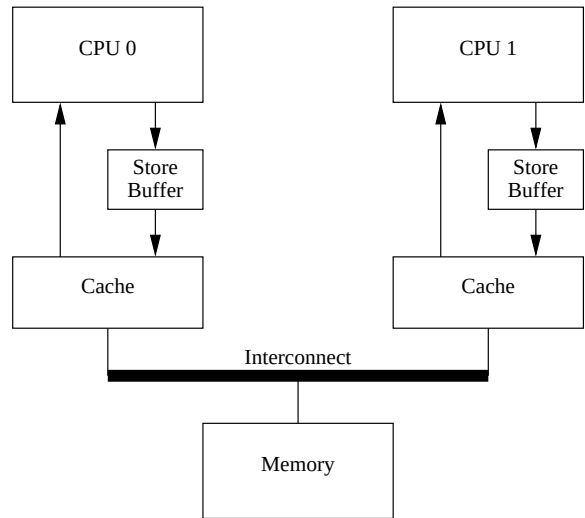


图5：带有存储缓冲区的缓存

```

1  a = 1;
2  b = a + 1;
3  assert(b == 2);

```

人们不会预期该断言会失败。然而，如果有人愚蠢到使用图5所示的非常简单的架构，他们将会感到惊讶。这样的系统可能会遇到以下事件序列：

- 1. CPU 0 开始执行 `a=1`。
- 2. CPU 0 在缓存中查找 “a”， 并发现

3. 因此，CPU 0 发送一个“读取无效”的消息，以获得包含“a”的缓存行的独占所有权。 4. CPU 0 将对“a”的存储记录在其存储缓冲区中。 5. CPU 1 收到“读取无效”消息后，通过传输该缓存行并 从其缓存中移除该缓存行来响应。 6. CPU 0 开始执行 `b=a+1`。 7. CPU 0 从 CPU 1 处接收缓存行，此时“a”的值仍为零。 8. CPU 0 从其缓存中加载“a”，发现其值为零。 9. CPU 0 将存储队列中的条目应用到新到达的缓存行上，将其缓存中“a”的值设置为一。

请提供需要翻译的英文学术术语列表。目前提供的信息中缺少具体的存储到包含“b”的缓存行中（假设该缓存行已被CPU 0 所拥有）。

请提供需要翻译的英文学术术语列表。目前提供的信息中缺少具体的

我们有两个“a”的副本，一个在缓存中，另一个在存储缓冲区中。
这个例子打破了非常重要的一条保证，即每个CPU总是会看到自己的操作仿佛是按程序顺序发生的。

这种保证对于软件类型来说是极其违背直觉的，以至于硬件工程师出于同情实现了“存储转发”，即每个CPU在执行加载操作时会参考（或称为“监听”）其存储缓冲区以及缓存，如图6所示。换句话说，某个CPU的存储操作可以直接转发到后续的加载操作中，而无需通过缓存传递。

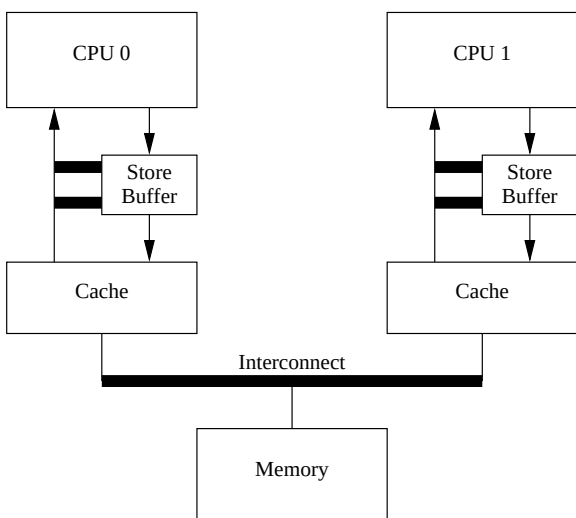


图6：带有存储转发的缓存

启用store转发后，上述序列中的第8项会在存储缓冲区中找到变量“a”的正确值1，因此最终变量“b”的值将会是2，如预期的那样。

```

1 void foo(void)
2 {
3     a = 1;
4     b = 1;
5 }
6
7 void bar(void)
8 {
9     while (b == 0) continue;
10    assert(a == 1);
11 }
  
```

假设CPU 0执行foo()，而CPU 1执行bar()。进一步假设包含“a”的缓存行仅存在于CPU 1的缓存中，包含“b”的缓存行由CPU 0拥有。那么操作序列可能是以下这样的：

1. CPU 0 执行 `a=1`。由于缓存行不在CPU 0的缓存中，因此CPU 0将“a”的新值放入其存储缓冲区，并发送一个“读取失效”消息。
2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。
3. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），所以它将“b”的新值存储在其缓存行中。
4. CPU 0 收到“读取”消息，并将其包含已更新的“b”值的缓存行发送给CPU 1，在其自己的缓存中标记该行作为“共享”。
5. CPU 1 接收包含“b”的缓存行并将其安装在其缓存中。
6. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。
7. CPU 1 执行 `assert(a==1)`，但由于其使用的是旧的“a”值，此断言失败。
8. CPU 1 收到“读取失效”消息，并将其包含“a”的缓存行发送给

3.3 Store Buffers and Memory Barriers

为了看到第二个复杂情况，即全局内存顺序的违反，请考虑以下代码序列，其中变量“a”和“b”初始值为零：

CPU 0 将其缓存中的这条缓存行置为无效。但为时已晚。

9. CPU 0 接收到了包含“a”的缓存行，并及时应用了缓冲存储操作，恰好成为CPU 1失败断言的受害者。

硬件设计师在此无能为力，因为CPU无法确定哪些变量是相关的，更不用说它们是如何关联的。因此，硬件设计师提供了内存屏障指令，允许软件向CPU告知这些关系。程序片段必须更新以包含内存屏障：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }
```

内存屏障 `smp_mb()` 将导致 CPU 在应用后续存储操作之前清空其写缓冲区。CPU 可以选择简单地暂停执行，直到写缓冲区为空后再继续，或者使用写缓冲区来暂存后续的存储操作，直至先前的所有条目都被应用到缓存行中。

采用后一种方法，操作序列可能是以下这样的：

1. CPU 0 执行 `a=1`。由于缓存行不在CPU 0的缓存中，因此CPU 0将“a”的新值放入其存储缓冲区，并发送一条“读取失效”消息。2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一条“读取”消息。3. CPU 0 执行 `smp_mb()`，并标记所有当前的存储缓冲区条目（即，`a=1`）。

4. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），但存储缓冲区中有一个标记条目。因此，CPU 0 不将新值“b”存储在缓存行中，而是将其放入存储缓冲区（但在一个 *unmarked* 条目中）。

CPU 0 收到“读取”消息，并将包含变量“b”原始值的缓存行传输给 CPU 1。它还将自己持有的该缓存行标记为“共享”。

6. CPU 1 接收包含“b”的缓存行并将其安装到自己的缓存中。

7. CPU 1 现在可以完成执行 `while(b==0)continue`，但由于发现“b”的值仍然是 0，它重复执行了 `while` 语句。新的“b”值安全地隐藏在 CPU 0 的存储缓冲区中。8. CPU 1 收到“读取失效”消息，并将包含“a”的缓存行传输给 CPU 0 并从自己的缓存中使该缓存行失效。9. CPU 0 接收包含“a”的缓存行并应用了存储缓冲区中的写操作。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的缓冲区中唯一的条目，因此CPU 0也可以存储“b”的新值——除非包含“b”的缓存行现在处于“共享”状态。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的“b”的缓存行置为无效，并向CPU 0 发送“确认”消息。

请提供需要翻译的英文学术文本内容，目前提供的信息中缺少具体的“”的缓存行不在其缓存中。因此，它向CPU 0 发送一个“读取”消息。

14. CPU 0 收到“确认”消息，并将包含“b”的缓存行置为“独占”状态。CPU 0 现在将“b”的新值存储到该缓存行中。

15. CPU 0 收到“读取”消息，并将包含变量“b”原始值的缓存行传输给CPU 1。它还将自己持有的该缓存行标记为“共享”。

16. CPU 1 接收包含“b”的缓存行并将其安装到自己的缓存中。

CPU 1 现在可以完成执行 `while(b==0) continue`，并且由于它发现变量“b”的值为 1，因此继续执行下一条语句。

18. CPU 1 执行了 `assert(a==1)`，但包含“a”的缓存行已不在其缓存中。一旦从CPU 0 获取该缓存行后，它将使用最新的“a”值，并且断言因此通过验证。

如您所见，这个过程涉及大量的记录工作。即使是直观上看似简单的操作，比如“加载a的值”，在硅片实现中也可能包含许多复杂的步骤。

4 Store Sequences Result in Unnecessary Stalls

不幸的是，每个存储缓冲区必须相对较小，这意味着执行一串适度长度的存储指令时，CPU可能会填满其存储缓冲区（例如，如果所有这些指令都导致缓存未命中）。此时，CPU必须再次等待无效操作完成以清空其存储缓冲区，才能继续执行。这种情况也可能在内存屏障之后立即出现，当`all`后续的存储指令必须等待无效操作完成，无论这些存储指令是否导致缓存未命中。

这种情况可以通过使无效化确认消息更快到达来改善。一种实现方法是使用每个CPU的无效化消息队列，或称为“无效化队列”。

4.1 Invalidate Queues

无效确认消息耗时较长的一个原因是，它们必须确保对应的缓存行确实被清除了。如果缓存忙于处理数据（例如，CPU正在进行大量数据的加载和存储操作），这些清除操作可能会被延迟。此外，在短时间内收到大量无效消息的情况下，某个CPU可能会落后于处理这些消息，从而可能导致所有其他CPU都陷入停滞状态。

然而，CPU并不需要在发送确认之前实际使缓存行失效。它也可以将失效消息排队，理解为该消息将在CPU发送有关该缓存行的任何进一步消息之前被处理。

4.2 Invalidate Queues and Invalidate Acknowledge

图7展示了一个带有无效队列的系统。一个配备了无效队列的CPU可以在收到无效消息时立即将其放入队列中进行确认，而无需等待对应的缓存行实际被无效化。当然，在准备发送无效消息之前，CPU必须检查其无效队列——如果对应缓存行的条目存在于无效队列中，则CPU不能立即传输该无效消息；它必须等到无效队列中的该项被处理完毕后才能继续。

将条目放入无效队列本质上是CPU的一种承诺，即在传输任何与该缓存行相关的MESI协议消息之前处理该条目。只要相应的数据结构不会高度竞争，CPU很少会因这种承诺而受到困扰。

然而，无效消息可以在无效队列中缓存这一事实为内存乱序访问提供了额外的机会，如下一节所述。

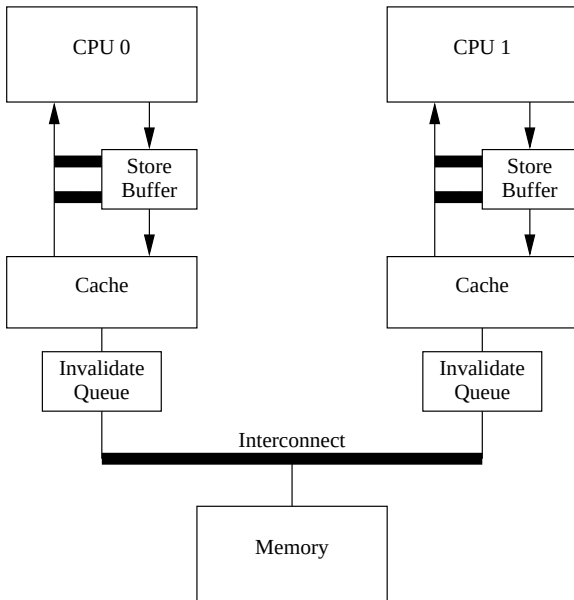


图7: 带有无效队列的缓存

4.3 Invalidate Queues and Memory Barriers

假设变量“a”和“b”的初始值均为零，“a”被只读复制（MESI共享状态），而“b”由CPU 0独占（MESI专有或修改状态）。然后假定CPU 0执行代码片段中的foo()，而CPU 1执行函数bar()：

```

1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    assert(a == 1);
12 }

```

然后操作序列可能是如下所示：

低频：

CPU 0 执行 `a=1`。在 CPU 0 的缓存中，对应的缓存行是只读的，因此 CPU 0 将“a”的新值放入其存储缓冲区，并发送一个“无效”消息以从 CPU 1 的缓存中清除相应的缓存行。

2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。

3. CPU 0 执行 `b=1`。它已经拥有此缓存行（换句话说，该缓存行已经在“修改”或“独占”状态），因此它将其缓存行中的“b”的新值存储下来。

4. CPU 0 收到“读取”消息，并将包含已更新值“b”的缓存行传输给CPU 1，同时在自己的缓存中标记该行状态为“共享”。5. CPU 1 收到针对“a”的“失效”消息，将其放入失效队列中，并向CPU 0 发送一个“失效确认”消息。需要注意的是，“a”的旧值仍然保留在CPU 1的缓存中。6. CPU 1 接收包含“b”的缓存行并安装到自己的缓存中。7. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。8. CPU 1 执行 `assert(a==1)`，但由于“a”的旧值仍然保留在CPU 1的缓存中，此断言失败。9. CPU 1 处理队列中的“失效”消息，并从自己的缓存中清除包含“a”的缓存行。但此时已经太迟了。

CPU 0 收到了来自自身的“无效确认”消息（针对“A”），因此刚刚缓冲的存储操作正好赶在了CPU 1 的失败断言之下，成了受害者。

再次，CPU设计者对此无能为力，因为硬件无法知晓那些对CPU而言只是不同位堆栈之间的关系。然而，内存屏障指令可以与失效队列交互，因此当某个CPU执行内存屏障时，它会标记当前存在于其失效队列中的所有条目，并强制后续的任何加载操作等待，直到所有已标记的条目都被应用到该CPU的缓存中。因此，我们可以添加如下所示的内存屏障：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_mb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_mb();
12    assert(a == 1);
13 }
```

随着这一改动，操作序列可能如下所示：

1. CPU 0 执行 `a=1`。在 CPU 0 的缓存中，包含“a”的缓存行是只读的，因此 CPU 0 将“a”的新值放入其存储缓冲区，并发送一个“无效”消息以清除 CPU 1 缓存中的相应缓存行。
2. CPU 1 执行 `while(b==0)continue`，但包含“b”的缓存行不在其缓存中。因此它发送一个“读取”消息。
3. CPU 0 执行 `b=1`。它已经拥有此缓存行（也就是说，该缓存行已经在“修改”或“独占”状态），所以它将“b”的新值存储在其缓存行中。
4. CPU 0 收到“读取”消息，并将其包含已更新的“b”值的缓存行发送给 CPU 1，在其自身的缓存中标记该行作为“共享”。

5. CPU 1 收到针对“a”的“无效”消息，将其放入无效队列，并向 CPU 0 发送一个“无效确认”消息。需要注意的是，“a”的旧值仍然保留在 CPU 1 的缓存中。
6. CPU 1 接收到包含“b”的缓存行并将其安装到自己的缓存中。
7. 现在，CPU 1 可以完成执行 `while(b==0)` 和 `continue`。由于它发现“b”的值为1，因此继续执行下一条语句。
8. CPU 1 执行 `smp_mb()`，在其无效队列中标记该条目。
9. 当前，CPU 1 执行 `assert(a==1)`，但由于其无效队列中包含针对“a”缓存行的标记条目，因此必须暂停此加载操作，直到该条目被应用为止。
10. CPU 1 处理“无效”消息，从其缓存中移除包含“a”的缓存行。
11. 现在，CPU 1 可以加载“a”的值了。但由于这导致缓存未命中，它必须发送一个“读取”消息来获取相应的缓存行。
12. CPU 0 收到来自 CPU 1 的针对“a”的“无效确认”消息，并因此应用缓冲存储操作，将相应缓存行的MESI状态更改为“修改”。
13. CPU 0 接收到来自 CPU 1 针对“a”的“读取”请求，并因此更改了相应缓存行的状态为“共享”，并将该缓存行传输给 CPU 1。
14. CPU 1 收到包含“a”的缓存行，现在可以执行加载操作。由于此加载返回更新后的“a”的值，断言通过验证。

经过大量MESI消息的传递，CPU最终得出了正确的结果。

5 Read and Write Memory Barriers

在上一节中，内存屏障被用来标记存储缓冲区和无效队列中的条目。但在我们的代码片段中，`foo()` 没有必要与无效队列进行任何操作，类似地，`bar()` 也没有必要与存储队列进行任何操作。

许多CPU架构因此提供了较弱的内存屏障指令，这些指令只执行上述两种操作中的某一种。大致来说，“读内存屏障”仅标记无效队列，“写内存屏障”仅标记存储缓冲区，而完整的内存屏障则同时进行这两项操作。

这种效果是，读内存屏障仅对执行它的CPU上的加载操作进行排序，因此所有在读内存屏障之前的加载操作都将看似已经完成，然后再执行任何后续的加载操作。同样地，写内存屏障仅对执行它的CPU上的存储操作进行排序，并且所有在写内存屏障之前的存储操作都将看似已经完成，然后再执行任何后续的存储操作。一个完整的内存屏障则同时对加载和存储操作进行排序，但仍然只针对执行该内存屏障的CPU。

如果我们将 `foo` 和 `bar` 更新为使用读写内存屏障，则它们将变为如下形式：

```
1 void foo(void)
2 {
3     a = 1;
4     smp_wmb();
5     b = 1;
6 }
7
8 void bar(void)
9 {
10    while (b == 0) continue;
11    smp_rmb();
12    assert(a == 1);
13 }
```

一些计算机甚至具有更多种类的内存屏障，但理解这三种变体将了解内存屏障提供良好的入门介绍。

通用。

6 Example Memory-Barrier Sequences

本节介绍了一些引人注目但微妙地存在问题的内存屏障用法。尽管其中许多方法在大多数情况下都能正常工作，某些甚至会在特定的CPU上始终如一地运行，但如果目标是编写能够在所有CPU上可靠运行的代码，则必须避免使用这些方法。为了更好地揭示这种细微的问题，我们首先需要关注一个敌对顺序的架构。

6.1 Ordering-Hostile Architecture

保罗遇到过多种敌对有序性的计算机系统，但这种敌对性总是极其微妙，理解它需要详细了解特定的硬件。为了避免针对某个具体的硬件供应商，并且作为吸引读者的一种替代方案，而不是让他们陷入详细的规格说明中，让我们设计一种神话般的、但却具有最大内存顺序敌对性的计算机架构。⁴

该硬件必须遵守以下顺序约束条件 [16, 17]：

每个CPU总是会将其自身的内存访问视为按程序顺序发生的。

2. CPU仅在两个操作引用不同位置时，才会重新排序给定的存储操作。

3. 给定CPU的所有在读内存屏障（`smp_rmb()`）之前的加载操作，都将被所有其他CPU感知为早于该读内存屏障之后的任何加载操作。

⁴Readers preferring a detailed look at real hardware architectures are encouraged to consult CPU vendors' manuals [19, 1, 8, 6, 15, 20, 10, 9, 13] or Gharachorloo's dissertation [4].

CPU 0	CPU 1	CPU 2
a=1; smp_wmb(); b=1;	while(b==0); c=1;	z=c; smp_rmb(); x=a; assert(z==0 x==1);

表2：内存屏障示例1

4. 给定CPU在写内存屏障（smp_wmb()）之前的所有存储操作，都将被所有其他CPU感知为发生在该写内存屏障之后的任何存储操作之前。

5. 给定CPU的所有访问操作（加载和存储）在全内存屏障（smp_mb()）之前进行，所有其他CPU都会感知到这些操作发生在任何跨越该内存屏障的操作之前。

Quick Quiz 6: 每个CPU都能按顺序看到自己的内存访问，这是否也保证了每个用户级线程能按顺序看到自己的内存访问？为什么或者为什么不呢？

想象一个大型非均匀缓存架构（NUCA）系统，为了在给定节点内为各个CPU提供公平的互连带宽分配，在每个节点的互连接口中提供了针对每颗CPU的队列，如图8所示。尽管某个特定CPU的访问顺序由该CPU执行的内存屏障所指定，但一对特定CPU之间的相对访问顺序可能会被严重重排，这一点我们将在后续内容中看到。

6.2 Example 1

图2展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。“a”、“b”和“c”初始值均为零。

⁵Any real hardware architect or designer will no doubt be loudly calling for Ralph on the porcelain intercom, as they just might be just a bit upset about the prospect of working out which queue should handle a message involving a cache line that both CPUs accessed, to say nothing of the many races that this example poses. All I can say is “Give me a better example”.

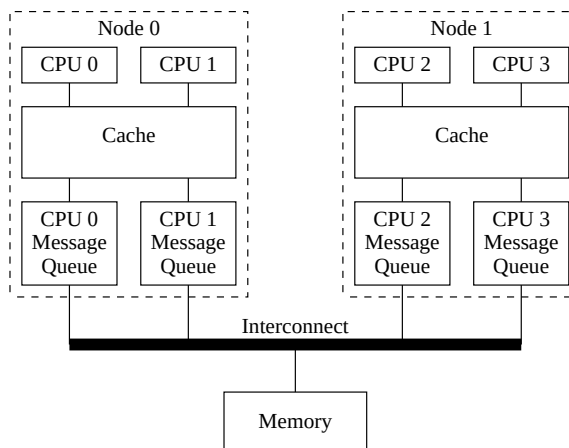


图8：示例敌序架构

假设CPU 0最近经历了许多缓存未命中，因此其消息队列已满，而CPU 1一直在缓存中运行，所以它的消息队列是空的。那么，CPU 0对“a”和“b”的赋值会立即出现在节点0的缓存中（从而对CPU 1可见），但会被阻塞在CPU 0之前的流量之后。相比之下，CPU 1对“c”的赋值将通过其之前为空的队列顺利传递。因此，CPU 2可能会先看到CPU 1对“c”的赋值，而看不到CPU 0对“a”的赋值，从而触发断言，尽管存在内存屏障。

在理论上，可移植代码无法使用这种编码技术，然而，在实际应用中，它确实可以在所有主流计算机系统上运行。

Quick Quiz 7: 在这段代码中，通过在CPU 1的“while”循环和对变量“c”的赋值之间插入一个内存屏障能否解决问题？为什么可以或不可以？

6.3 Example 2

图3展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。“a”和“b”初始值均为零。

再次假设CPU 0最近经历了许多缓存未命中，因此其消息队列已满，而CPU 1则一直独立运行在区域内。

CPU 0	CPU 1	CPU 2
a=1;	while(a==0); smp_mb(); b=1;	y=b; smp_rmb(); x=a; assert(y==0 x==1);

表3：内存屏障示例2

	CPU 0	CPU 1	CPU 2
1	a=1;		
2	smb_wmb();		
3	b=1;	while(b==0);	while(b==0);
4		smp_mb();	smb_mb();
5		c=1;	d=1;
6	while(c==0);		
7	while(d==0);		
8	smp_mb();		
9	e=1;		assert(e==0 a==1);

表4：内存屏障示例3

缓存中的消息队列为空。此时，CPU 0 对变量 “a” 和 “b” 的赋值会立即出现在节点0的缓存中（因此对 CPU 1可见），但会被CPU 0之前的通信阻塞。相比之下，CPU 1 对变量 “b” 的赋值将通过其之前空的消息队列顺利传递。因此，CPU 2 可能会在看到 CPU 0 对 “a” 的赋值之前先看到 CPU 1 对 “b” 的赋值，从而触发断言，尽管存在内存屏障。

在理论上，这种编码技术不可用于便携式代码，然而实际上，它确实可以在所有主流计算机系统上运行。

6.4 Example 3

图4展示了三个代码片段，这些代码片段由CPU 1、CPU 2和CPU 3并发执行。所有变量初始值均为零。

注意，直到看到CPU 0在第3行对 “b” 的赋值，CPU 1和CPU 2都无法执行到第4行。一旦CPU 1和CPU 2在第3行执行了内存屏障指令，它们都将保证能看到所有发生在CPU 0的第2行内存屏障之前的赋值操作。同样地，CPU 0的内存屏障也确保了这一点。

在第8行与CPU 1和CPU 2在第4行的操作配对，因此CPU 0不会在它的对 “a” 的赋值可见于其他两个CPU之前执行对 “e” 的赋值操作。因此，CPU 2在第9行的断言必定会触发 *not*。

Quick Quiz 8: 假设CPU 1和CPU 2的第3至第5行代码位于中断处理程序中，而CPU 2的第9行在进程级别运行。为了使代码能够正确工作，即防止断言触发，需要做哪些修改？



Linux 内核的 `synchronize_rcu()` 原语使用了与该示例中所示类似的算法。

7 Memory-Barrier Instructions For Specific CPUs

每个CPU都有其独特的内存屏障指令，这使得跨平台移植变得颇具挑战性，如表5所示。事实上，许多软件环境，包括

threads和Java，都禁止直接使用内存屏障，仅允许程序员使用包含必要内存屏障的互斥原语。在表格中，前四列显示了给定CPU是否允许四种可能的加载和存储指令重排序组合。接下来的两列则表明给定CPU是否允许加载和存储指令与原子指令之间进行重排序。仅通过六种CPU，我们就能发现五种不同的加载-存储重排序组合，并且有三种可能的原子指令重排序组合。

第七列 “依赖读取重排序” 需要一些解释，这将在接下来讨论Alpha CPU的部分中进行说明。简而言之，Alpha要求对链式数据结构的读者和更新者都使用内存屏障。是的，这意味着Alpha确实可以在获取指向它的指针之前就获取该指针所指向的数据，虽然听起来很奇怪但这是事实。如果您认为我在编造这些内容，请参见：http://www.openvms.compaq.com/wizard/wiz_2637.html。这种极其弱化的内存模型的好处在于，Alpha可以

	sd aol r e l A d e r e d r o e R s d a o L	se r o l S r e l A d e r e d r o e R s d a o L	se r o l S r e l A d e r e d r o e R se r o l S	sd aol r e l A d e r e d r o e R se r o l S	sd aol h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A	se r o l S h l W d e r e d r o e R s n o i c u r t s n l c i m o l A
Alpha	Y	Y	Y	Y	Y	Y	Y	Y	Y
AMD64				Y					
IA64	Y	Y	Y	Y	Y	Y	Y		Y
(PA-RISC)	Y	Y	Y	Y					
PA-RISC CPUs									
POWER ^{1.0}	Y	Y	Y	Y	Y	Y			Y
(SPARC RMO)	Y	Y	Y	Y	Y	Y			Y
(SPARC PSO)			Y	Y		Y			Y
SPARC TSO				Y					Y
x86				Y					Y
(x86 OOSTore)	Y	Y	Y	Y					Y
zSeries ⁶⁴				Y					Y

表5：内存顺序概要

使用更简单的缓存硬件，这反过来使得在Alpha架构的鼎盛时期能够实现更高的时钟频率。

最后一列表明给定的CPU是否具有不一致的指令缓存和流水线。此类CPU在执行自修改代码时需要特殊指令。

括号中的CPU名称表示在架构上允许但实际很少使用的模式。

对于内存屏障采取的“直接拒绝”方法在某些情况下是完全合理的，但在诸如Linux内核这样的环境中，则需要直接使用内存屏障。因此，Linux提供了一套精心挑选的、最低公倍数级别的内存屏障原语。

以下为翻译后的最终文本：目标如下：

- `smp_mb()` “内存屏障”，它规定了加载和存储操作的顺序。这意味着，在内存屏障之前的加载和存储操作将会在任何后续的加载和存储操作之前被提交到内存中。
- `smp_rmb()` “仅排序加载操作的读内存屏障”。
- `smp_wmb()` “仅排序存储操作的写内存屏障”。
- `smp_read_barrier_depends()` 这迫使依赖于先前操作的后续操作按顺序执行。此原语仅在Alpha平台上不是空操作。
- `mmiowb()` 这会强制对受全局自旋锁保护的MMIO写操作进行排序。在所有平台上的内存屏障已通过自旋锁强制执行MMIO顺序的情况下，此原语为无操作。具有非无操作定义`mmiowb()`的平台包括某些（但并非全部）IA64、FRV、MIPS和SH系统。由于该原语相对较新，因此利用它的驱动程序相对较少。

`smp_mb()`、`smp_rmb()` 和 `smp_wmb()` 原语也迫使编译器避免采用任何会重新排序内存访问优化的优化策略，这些优化可能会跨越屏障。类似地，`smp_read_barrier_depends()` 原语在Alpha CPU上也有相同的效果。

这些原语仅在SMP内核中生成代码，然而每个也有一个单处理器版本（`smp_mb()`、`tt_smp_rmb()`、`smp_wmb()`和`smp_read_barrier_depends()`），即使在UP内核中也会生成内存屏障。通常应使用`smp`版本。然而，在编写驱动程序时，这些后者的原语非常有用，因为MMIO访问必须保持有序，即便是在UP内核中也是如此。如果没有内存屏障指令，CPU和编译器都会乐意重新排列这些访问操作，这至少会导致设备行为异常，并且可能会导致你的内核崩溃，甚至在某些情况下损坏硬件。

因此，大多数内核程序员无需担心每一种CPU特有的内存屏障细节，只要他们遵守这些接口即可。当然，如果你在特定CPU架构的底层代码中工作，则另当别论。

此外，Linux的所有锁原语（自旋锁、读写锁、信号量、RCU等）都包含了所需的屏障原语。因此，如果你正在处理使用这些原语的代码，则无需担心Linux的内存顺序原语。

话说，深入了解每个CPU的内存一致性模型在调试时非常有帮助，更不用说编写特定架构的代码或同步原语时。

此外，他们说一点知识是非常危险的。试想一下，拥有大量知识会造成多大的破坏！对于那些希望更深入理解个别CPU内存一致性模型的人来说，接下来的部分将描述最流行和重要的CPU的相关内容。虽然没有什么能替代实际阅读特定CPU的文档，但这些部分可以提供一个很好的概览。

7.1 Alpha

说太多关于一款已经宣布停产的CPU似乎有些奇怪，但Alpha处理器却很有研究价值，因为它具有最弱的内存排序模型，并且会最为激进地重排内存操作。因此，它定义了适用于所有CPU（包括Alpha）的Linux内核内存排序原语。对于Linux内核开发者来说，理解Alpha处理器的重要性因此显得出乎意料地重要。

Alpha与其它CPU之间的区别如图9所示的代码所阐明。该代码第9行的`smp_wmb()`确保了在第6至8行进行的元素初始化会在该元素于第10行被添加到列表之前执行，从而使无锁搜索能够正确工作。也就是说，在所有CPU上（除了Alpha之外），都做出了这样的保证。

Alpha的内存顺序极为弱化，因此图9中第20行的代码可能会看到初始化前（第6至第8行）存在的旧垃圾值。

```
1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         /* BUG ON ALPHA!!! */
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }
```

图9：插入与无锁查找

图10展示了一个高度并行的机器上，当缓存被分区时这种情况是如何发生的。假设列表头`head`将由缓存银行0处理，而新元素则由缓存银行1处理。在Alpha架构中，`smp_wmb()`可以保证图9中的第6至8行执行的缓存失效操作会先于第10行的操作到达互连网络，但对新值到达读取CPU核心的顺序没有任何保证。例如，有可能读取CPU的缓存银行1非常繁忙，而缓存银行0则处于空闲状态。这可能导致针对新元素的缓存失效被延迟，从而使读取CPU获取到指针的新值，但却看到的是新元素的老缓存值。请参阅之前提到的网站以获得更多信息，或者如果您认为我只是在编造这些内容，请再次查看。⁶

可以放置一个 `smp_rmb()` 原始元素来

⁶Of course, the astute reader will have already recognized that Alpha is nowhere near as mean and nasty as it could be, the (thankfully) mythical architecture in Section 6.1 being a case in point.

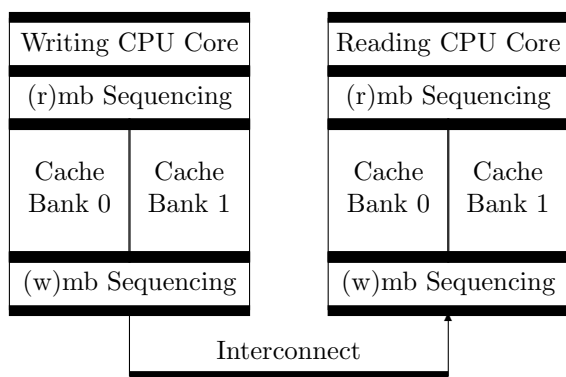


图10: 为什么SMP读屏障依赖关系是必要的

在指针获取和解引用之间。然而，这给了一些系统（如i386、IA64、PPC和SPARC）带来了不必要的开销，这些系统在读取侧会尊重数据依赖性。Linux 2.6内核中添加了一个`smp_read_barrier_depends()`原语来消除这些系统的开销。此原语可以在图11的第19行中看到如何使用。

也可以实现一种软件屏障，用以替代`smp_wmb()`，强制所有读取CPU按顺序看到写入CPU的写操作。然而，Linux社区认为这种方法在Alpha等极弱有序的CPU上会带来过高的开销。这种软件屏障可以通过向其他所有CPU发送跨处理器中断（IPI）来实现。当收到这样的IPI时，一个CPU将执行内存屏障指令，实施内存屏障清理操作。为了避免死锁，还需要额外的逻辑处理。当然，尊重数据依赖关系的CPU会将此类屏障简单定义为`smp_wmb()`。随着Alpha逐渐退出历史舞台，或许未来应重新考虑这一决定。

Linux的内存屏障原语名称源自Alpha指令，因此`smp_mb()`是`{smp_mb()}`，`smp_rmb()`是`{smp_rmb()}`，而`smp_wmb()`是`{smp_wmb()}`。Alpha架构中

```
1 struct el *insert(long key, long data)
2 {
3     struct el *p;
4     p = kmalloc(sizeof(*p), GFP_ATOMIC);
5     spin_lock(&mutex);
6     p->next = head.next;
7     p->key = key;
8     p->data = data;
9     smp_wmb();
10    head.next = p;
11    spin_unlock(&mutex);
12 }
13
14 struct el *search(long key)
15 {
16     struct el *p;
17     p = head.next;
18     while (p != &head) {
19         smp_read_barrier_depends();
20         if (p->key == key) {
21             return (p);
22         }
23         p = p->next;
24     };
25     return (NULL);
26 }
```

图11: 安全插入与锁-free搜索

仅在`smp_read_barrier_depends()`是一个`smp_mb()`而非空操作的CPU中。

参见文献[19]以获取更多关于Alpha的详细信息。

7.2 AMD64

AMD64与x86兼容，并已最近更新了其内存模型[2]，以强制执行实际实现长期以来提供的更严格的顺序。Linux在AMD64上的`smp_mb()`原语的实现为`mfence`，`smp_rmb()`为`lfence`，而`smp_wmb()`为`sfence`。理论上这些可能被放宽，但任何此类放宽都必须考虑SSE和3DNow指令的影响。

7.3 IA64

IA64提供了一种弱一致性模型，因此在没有显式内存屏障指令的情况下，IA64有权任意重排内存引用[8]。IA64有一个名为`mf`的内存屏障指令，但也为加载、存储以及某些原子操作提供了“半内存屏障”修饰符。



图12：半内存屏障

指令[7]。acq修饰符防止后续的内存引用指令被重新排序到acq之前，但允许之前的内存引用指令在acq之后进行重排。类似地，rel修饰符防止之前的内存引用指令被重新排序到rel之后，但允许后续的内存引用指令在rel之前进行重排。

这些半内存栅栏在临界区中非常有用，因为可以安全地将操作推进到临界区内，但允许它们溢出则可能致命。然而，作为少数具有此特性的CPU之一，IA64定义了Linux中与锁获取和释放相关的内存顺序语义。

IA64 v6 指令用于 Linux 内核中的 v7、v8 和 v9 原语。哦，尽管有传言称不然，“mf”助记符确实代表“内存栅栏”。

最终，IA64为“释放”操作提供了全局总顺序，包括“mf”指令。这提供了一种传递性的概念，即如果一段给定的代码片段看到某个访问已经发生，则任何后续的代码片段也将看到该早期访问已发生。假设所有涉及的代码片段都正确使用了内存屏障。

7.4 PA-RISC

尽管PA-RISC架构允许完全重新排序加载和存储操作，但实际上的CPU是全序执行的[14]。这意味着Linux内核中的内存顺序原语不会生成任何代码，然而它们确实使用了gcc memory属性来禁用编译器优化，这些优化可能会在内存屏障两侧重排代码。

7.5 POWER

POWER 和 PowerPC® 系列处理器具有多种内存屏障指令 [6, 15]:

sync 导致所有先前的操作必须完全完成，才能开始任何后续操作。因此，这条指令相当昂贵。

轻量级同步指令会对后续的加载和存储操作进行排序，同时也对存储操作进行排序。然而，它不会对后续的加载操作与当前的存储操作之间进行排序。有趣的是，该指令执行的排序规则与zSeries系统以及SPARC TSO一致。

3. 如你所想，eieio (enforce in-order execution of I/O)会确保所有之前的可缓存存储操作在所有后续的存储操作之前完成。然而，对可缓存内存的存储操作与对不可缓存内存的存储操作是分别排序的，这意味着eieio不会强制一个MMIO存储操作先于自旋锁释放操作执行。

4. isync强制所有先前的指令在任何后续指令开始执行之前必须看起来已经完成。这意味着之前的指令必须足够进展，以确保它们可能引发的所有异常（如陷阱）要么已经发生，要么可以保证不会发生，并且这些指令所产生的所有副作用（例如页表更改）会被后续指令观察到。

不幸的是，这些指令与Linux的wmb()原语并不完全对应，后者要求

all 存储操作需要有序执行，但不需要其他高开销的操作。然而别无选择：ppc64版本的wmb()和mb()被定义为重量级的sync指令。但是，Linux中的smp_wmb()指令从未用于MMIO（毕竟，在UP和SMP内核中，驱动程序必须仔细地MMIO进行排序），因此它被定义为轻量级的eieio指令。这条指令可能是唯一一个具有五个元音字母助记符的指令，其含义是“强制I/O按顺序执行”。同样，smp_mb()指令也被定义为sync指令，但smp_rmb()和rmb()都被定义为轻量级的lwsync指令。

功率特性具有“累积性”，可以用来获得传递性。当使用得当时，任何看到早期代码片段结果的代码也将看到该早期代码片段本身访问的内容。更多细节可参见 McKenney 和 Silvera [18]。

许多POWER架构的成员具有非一致性的指令缓存，因此对内存中的存储操作不一定反映在指令缓存中。值得庆幸的是，如今很少有人编写自修改代码，但JIT编译器和传统编译器却经常这样做。此外，重新编译最近运行过的程序从CPU的角度来看就像自修改代码一样。icbi指令（指令缓存块无效化）会将指定的缓存行从指令缓存中失效，并且可以在这些情况下使用该指令。

7.6 SPARC RMO, PSO, and TSO

Solaris在SPARC架构上使用TSO（全存.Store顺序），而当Linux为“sparc”32位架构构建时也是如此。然而，64位的Linux内核（即“sparc64”架构）则以RMO（宽松内存顺序）模式运行SPARC [20]。SPARC架构还提供了一种中间级PSO（部分存.Store顺序）。任何在RMO下运行的程序也将在PSO或TSO下运行，同样地，任何在PSO下运行的程序也将在TSO下运行。然而，将一个共享内存并行程序向相反方向移动可能需要谨慎插入内存屏障，尽管如前所述，标准使用同步机制的程序可以避免这一问题。

内存操作原语无需担心内存屏障。

SPARC 提供了一个非常灵活的内存屏障指令[20]，允许对顺序进行细粒度控制：

StoreStore: 将前面的存储指令排序在随后的存储指令之前。（此选项由Linux smp_wmb()原语使用。）

LoadStore: 将前序的加载指令排序在后续的存储指令之前。

StoreLoad: 将前面的存储指令排序在后续的加载指令之前。

LoadLoad: 将先前的加载指令排序在后续的加载指令之前。（此选项由Linux smp_rmb()原语使用。）

Sync完成所有先前的操作后再开始后续的操作。

MemIssue完成先前的内存操作后再进行后续的内存操作，这对于某些内存映射I/O的情况非常重要。

Lookaside与MemIssue相同，但仅适用于先前的存储操作和后续的加载操作，并且仅针对访问同一内存位置的存储和加载操作。

Linux smp_mb()使用前四个选项的组合，如membar #LoadLoad | #LoadStore | #StoreStore | #StoreLoad所示，从而完全有序地执行内存操作。

因此，为什么需要membar #MemIssue? 因为一个membar #StoreLoad可能会允许后续的加载操作从写缓冲区获取值，如果这次写入的是具有副作用的MMIO寄存器，则这将是灾难性的。相比之下，membar #MemIssue会在允许加载操作执行之前等待写缓冲区被刷新，从而确保加载操作确实是从MMIO寄存器中获取值。驱动程序也可以使用membar #Sync，但在不需要更昂贵的membar #Sync所提供的额外功能的情况下，较轻量级的membar #MemIssue更为优选。

`membar #Lookaside` 是 `membar #MemIssue` 的轻量级版本，适用于向某个 MMIO 寄存器写入数据会直接影响该寄存器下次读取值的场景。然而，当对某个 MMIO 寄存器的写操作会影响另一个 *some other* MMIO 寄存器的下次读取值时，则必须使用更 *heavy weight* 的 `membar #MemIssue`。

不清楚为什么 SPARC 没有将 `wmb()` 定义为 `membar #MemIssue`，将 `smp_wmb()` 定义为 `membar #StoreStore`，因为当前的定义似乎容易受到某些驱动程序中错误的影响。很有可能所有运行 Linux 的 SPARC CPU 都实现了一种比架构允许的更为保守的内存排序模型。

SPARC 要求在指令被存储和执行之间使用一条 `flush` 指令[20]。这是为了从 SPARC 的指令缓存中清除该位置之前的任何值。注意，`flush` 需要一个地址，并且只会从指令缓存中清除指定的那个地址。在 SMP 系统上，所有 CPU 的缓存都会被刷新，但没有方便的方法来确定跨 CPU 刷新何时完成，尽管有一份实现说明提到这一点。

7.7 x86

由于 x86 处理器提供了“进程顺序”，确保所有 CPU 对特定 CPU 写入内存的操作顺序达成一致，因此 `smp_wmb()` 原语对于 CPU 而言是无操作的[10]。然而，需要编译器指令来防止编译器执行可能导致跨 `smp_wmb()` 原语重新排序的优化操作。

另一方面，x86 CPU 传统上对加载操作不提供任何顺序保证，因此 `smp_mb()` 和 `smp_rmb()` 原语展开为 `lock;addl`。这条原子指令既作为加载操作的屏障也作为存储操作的屏障。

最近，英特尔为 x86 架构发布了一套内存模型 [11]。结果发现，英特尔的实际 CPU 执行的顺序约束比之前规范中声明的更为严格，因此这套模型实际上只是强制规定了早期的事实上的行为。更近一些时候，英特尔发布了更新后的内存模型。

对于 x86 架构[12]，虽然要求全局存储操作遵循总序规则，但单个 CPU 仍可将其自身的存储操作视为早于该总序规则所指示的时间点发生。这一对总序的例外是为了允许重要的涉及存储缓冲区的硬件优化。软件可以使用原子操作来覆盖这些硬件优化，这也是为什么原子操作通常比非原子操作更昂贵的原因之一。这种全局存储顺序在较旧的处理器上是受 *not* 保证的。

然而，需要注意的是，某些 SSE 指令是弱有序的（`cflflush` 和非临时移动指令 [9]）。具有 SSE 功能的 CPU 可以使用 `mfence` 来实现 `smp_mb()`，使用 `lfence` 来实现 `smp_rmb()`，以及使用 `sfence` 来实现 `smp_wmb()`。

某些版本的 x86 处理器具有一个模式位，用于启用乱序存储，对于这些 CPU，`smp_wmb()` 也必须被定义为 `lock;addl`。

尽管许多较早的 x86 实现能够支持自修改代码而无需任何特殊指令，但新版本的 x86 架构不再要求 x86 CPU 必须如此宽容。有趣的是，这种放宽正好赶上了让即时编译器实现者感到不便的时候。

7.8 zSeries

zSeries 机器构成了 IBM 的大型机家族，此前被称为 360、370 和 390 系列[13]。尽管并行处理在 zSeries 中出现较晚，但考虑到这些大型机早在 20 世纪 60 年代中期就开始发货，这并不算什么。该指令用于 Linux 的原语，并且具有相对较强的记忆顺序语义，如表 5 所示，这应该允许原语成为一个，并且到您阅读本文时，这一变化可能已经发生。实际上，表格低估了情况，因为 zSeries 内存模型在其他情况下是顺序一致的，这意味着所有 CPU 都将同意来自不同 CPU 的无关存储指令的顺序。

与大多数 CPU 一样，zSeries 架构不保证缓存一致的指令流。

因此，自修改代码必须在更新指令和执行它们之间插入一条串行化指令。不过，许多实际的zSeries机器确实可以在没有串行化指令的情况下支持自修改代码。zSeries指令集提供了一组大量的串行化指令，包括比较并交换、某些类型的分支（例如前述的bcr 15,0指令）以及测试并设置等。

8 Are Memory Barriers Forever?

近期出现了一些系统，在整体上对乱序执行不那么激进，特别是在重新排序内存引用方面尤为明显。这一趋势是否会继续发展到内存屏障成为过去式的地步？

支持这一观点的论据会提到拟议中的大规模多线程硬件架构，在这种架构中，每个线程都会等待内存准备好，而在其间，其他数十、数百甚至数千个线程可以继续推进。在这种架构下，无需使用内存屏障，因为一个特定的线程只需在所有未完成的操作完成后才执行下一个指令。由于可能有成千上万个其他线程存在，CPU将被完全利用，因此不会浪费任何CPU时间。

反对的观点认为，能够扩展到一千个线程的应用程序数量极为有限，并且实时需求越来越严格，某些应用的实时响应要求已经达到数十微秒。现有的实时响应要求已经很难满足，而在大规模多线程场景下，单线程吞吐量极低的情况下，这一问题将变得更加严峻。

另一个论点认为，日益复杂的延迟隐藏硬件实现技术可能会使CPU在提供几乎全部乱序执行的性能优势的同时，还能提供完全顺序一致性的执行幻象。

一种反对观点认为，无论是由电池供电的设备还是环境保护责任都提出了越来越严格的能量效率要求。

谁对谁错我们无从得知，因此正在为两种可能的情况做准备。

9 Advice to Hardware Designers

硬件设计师可以采取多种措施来增加软件开发人员的工作难度。以下是我们在过去遇到的一些例子，希望这些经验能有助于预防未来出现类似问题：

忽略缓存一致性的I/O设备。

这种迷人的缺陷可能导致从内存发起的直接存储器访问（DMA）操作未能捕获输出缓冲区中的最近更改，或者更糟糕的是，在DMA完成之后立即将输入缓冲区的内容覆盖为CPU缓存中的内容。为了使系统在这种行为面前正常工作，您必须在将任何DMA缓冲区呈现给I/O设备之前，仔细地清空该缓冲区内所有位置的CPU缓存。即便如此，仍需小心避免指针错误，因为即使是对输入缓冲区的一次误读也可能导致数据输入损坏。

2. 忽略缓存一致性的设备中断。

这听起来似乎无伤大雅——毕竟，中断并不是内存引用，对吧？但设想一个具有分隔缓存的CPU，其中一部分非常繁忙，因此持有输入缓冲区的最后一段缓存行。如果相应的I/O完成中断到达该CPU，则该CPU对缓冲区最后一段缓存行的内存引用可能会返回旧数据，从而再次导致数据损坏，但在后续的崩溃转储中将无法察觉。等到系统开始转储有问题的输入缓冲区时，DMA很可能已经完成了。

3. 忽略缓存一致性的进程间中断（IPIs）。

这可能会出现問題，如果中断处理器指令（IPI）在其对应消息缓冲区中的所有缓存行都未提交到内存之前就已到达目的地。

4. 超前于缓存一致性的上下文切换。

如果内存访问可以过于无序地完成，那么上下文切换可能会非常糟糕。如果任务在所有对源CPU可见的内存访问到达目标CPU之前就在一个CPU和另一个CPU之间快速切换，那么该任务可能很容易看到相应的变量恢复到之前的值，这会严重混淆大多数算法。

5. 过于友好的模拟器和仿真器。

在实际编写模拟器或仿真器以强制内存重排序时会遇到困难，因此在这种环境中运行良好的软件首次在真实硬件上运行时可能会遭遇令人不悦的状况。不幸的是，目前硬件仍然比模拟器和仿真器更加复杂多变，但我们希望这种情况能够得到改善。

再次鼓励硬件设计师避免这些做法！

Acknowledgements

我感谢许多CPU架构师耐心地解释了他们CPU中的指令和内存重排序特性，特别是Wayne Cardoza、Ed Silha、Anton Blanchard、Brad Frey、Cathy May、Derek Williams、Tim Slegel、Juergen Probst、Ingo Adlung和Ravi Arimilli。Wayne尤其值得特别感谢，因为他耐心地解释了Alpha处理器对依赖性加载的重排序机制，而我对这一课的接受过程非常艰难。

Legal Statement

本文仅代表作者观点，不代表IBM的立场。

IBM、zSeries 和 Power PC 是国际商业机器 corporation 在美国及其他国家或地区的注册商标。Linux 是 Linus Torvalds 的注册商标。i386 是 Intel Corporation 或其子公司在美国及其他国家或地区的商标。其他公司产品和服务名称可能是相应公司的商标或服务标志。© 2005 IBM Corporation。

Answers to Quick Quizzes

Quick Quiz 1:

Answer: 如果两个CPU同时尝试使同一缓存行失效会发生什么？第一个获得共享总线访问权的CPU“获胜”。另一个CPU必须使其持有的该缓存行失效，并向获胜的CPU发送一个“失效确认”消息。当然，失败的CPU会立即发出一次“读取并失效”的事务请求，因此获胜CPU的优势将是短暂的。

Quick Quiz 2:

Answer: 当一个“无效”消息出现在大型多处理器系统中时，每个CPU都必须给出一个“无效确认”响应。这种大量的“无效确认”响应是否会完全饱和系统总线？

大规模多处理器系统如果确实那样实现的话，大型多处理器系统，尤其是非统一内存访问（NUMA）机器，通常会采用所谓的“目录式”缓存一致性协议来避免此类及其他问题。

Quick Quiz 3:

Answer: 如果SMP机器确实使用消息

随意而论，为何还要使用SMP呢？过去几十年来，这个问题一直存在不少争议。一种回答是缓存一致性协议相对简单，因此可以直接在硬件中实现，从而获得软件消息传递无法达到的带宽和低延迟。另一种回答则是从经济角度考虑，大型SMP机器与小型SMP集群的成本差异决定了其选择。还有一种回答是SMP编程模型比分布式系统更容易使用，但反驳者可能会指出高性能计算集群和MPI的出现。因此，争论仍在继续。

Quick Quiz 4:

Answer: 硬件是如何处理上述延迟过渡的？

通常通过添加额外的状态来实现这一点，尽管这些额外状态不必实际存储在缓存行中，因为一次只有少数几行会处于过渡状态。延迟转换的需要只是导致现实世界中的缓存一致性协议比本附录描述的简化的MESI协议复杂得多的问题之一。Hennessy和Patterson的经典计算机体系结构导论[5]涵盖了这些问题中的许多。

Quick Quiz 5:

Answer: 什么样的操作序列可以将所有CPU缓存都置回“无效”状态？

至少在CPU指令集中不存在特殊的“清空缓存”指令的情况下，这样的序列并不存在。不过大多数CPU确实拥有此类指令。

Quick Quiz 6:

Answer: 每个CPU都能按顺序看到自己的内存访问，这是否也保证了每个用户级线程能按顺序看到自己的内存访问？为什么或者为什么不呢？

考虑线程从一个CPU迁移到另一个CPU的情况，目标CPU感知到源CPU最近的内存操作出现乱序。为了保持用户模式下的正常状态，内核开发者必须在线程切换路径中使用内存屏障。然而，已经要求的锁定机制应该自动提供所需的内存屏障，以确保用户级任务能够按顺序看到自己的访问操作。也就是说，在设计超级优化调度器时（无论是内核层面还是用户层面），请务必牢记这一场景！

Quick Quiz 7:

Answer: 这段代码能否通过在CPU 1的“while”循环和对变量“c”的赋值之间插入一个内存屏障来修复？为什么不能？不可以。这样的内存屏障只会强制本地于CPU 1的顺序，不会影响CPU 0与CPU 1访问之间的相对顺序，因此断言仍然可能失败。然而，所有主流计算机系统都提供某种机制以实现“传递性”，即直观的因果顺序：如果B看到了A访问的效果，并且C看到了B访问的效果，则C也必须看到A访问的效果。

Quick Quiz 8:

Answer: 假设CPU 1和CPU 2的第3至5行代码处于中断处理程序中，而CPU 2的第9行在进程级别运行。为了使代码正确工作，即防止断言触发，需要做哪些修改？断言必须编写为确保“e”的加载先于“a”的加载进行。在Linux内核中，可以使用barrier()原语以与之前示例中的内存屏障类似的方式实现这一点。

References

【v11】。【v12】

chitecture Programmer's Manual Volumes 1-5, 2002.

【2】ADVANCED MICRO DEVICES.

AMD x86-64 Architecture Programmer's Manual Volume 2: System Programming, 2007.

【3】CULLER, D. E., SINGH, J. P., AND GUPTA, A. *Parallel Computer Architecture: a Hardware/Software Approach*. 劳伦斯·埃里森出版社, 1999.

Morgan Kaufmann 出版社的中文译名通常为“劳伦斯·埃里森出版社”，因为 Morgan Kaufmann 是由 Lawrence Halliday Ellison 创立的。如果需要保留原文出版社名称，可以改为“劳伦斯·埃里森出版社”。

【4】IBM MICROELECTRONICS AND MOTOROLA. *PowerPC Microprocessor Family: The Programming Environment*, 1995.

【5】HENNESSY, J. L., AND PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 施密特出版社, 1995.

Morgan Kaufmann 出版社的中文译名通常为“施密特出版社”。如果需要保留英文原文，可以改为：[5]

【6】IBM MICROELECTRONICS AND MOTOROLA. *PowerPC Microprocessor Family: The Programming Environment*, 1995.

【7】INTEL CORPORATION.

Intel Itanium Architecture Software Developer's Manual Volume 3: Instruction Set Reference, 2002.

【8】INTEL CORPORATION. *Intel Itanium Architecture Software Developer's Manual Volume 3: System Architecture*, 2002.

【9】INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 2B: Instruction Set Reference, N-Z*, 2004. 可在 <ftp://download.intel.com/design/Pentium4/manuals/25366714.pdf> 查阅 [查阅日期：2005年2月16日].

【10】INTEL CORPORATION. *IA-32 Intel Architecture Software Developer's Manual Volume 3: System Programming Guide*, 2004. 获取地址：<ftp://download.intel.com/design/>

<Pentium4/manuals/25366814.pdf> [查看日期：2005年2月16日]

【37】Intel 64 Architec-

ture Memory Ordering White Paper, 2007年。

网址：<http://developer.intel.com/products/processor/manuals/318147.pdf> [引用日期：2007年9月7日].

【12】INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developers Manual, Volume 3A: System Programming Guide, Part 1, 2* 009. 可在 <http://download.intel.com/design/processor/manuals/253668.pdf> 查阅 [查阅日期：2007年9月7日].

【参考文献】【13】IBM. *z/Architecture 操作原理*. 可在 <http://publibz.boulder.ibm.com/epubs/pdf/dz9zr003.pdf> 查阅 [引用日期：2005年2月16日]. 2004年5月。

【14】KANE, G. *PA-RISC 2.0 Architecture*. 惠普专业图书, 1996.

PowerPC存储模型与AIX编程. 引用日期：2005年1月31日。出版日期：2002年8月。

现代微处理器中的内存顺序，第一部分。IEEE Spectrum, 136 (2005年8月), 52-57. 可在 <https://www.spectrum.ieee.org/> [2007年11月30日访问] 查阅。

现代微处理器中的内存排序，第二部分。计算机架构新闻快报, 137 (2005年9月), 78-82页。可访问：<http://www.linuxdevcenter.com/pub/a/linux/2005/09/01/memory-ordering-part2.html> [引用日期：2007年11月30日]

示例电源实现用于c/c内存[v72][v73]

模型版本: <http://www.rdrop.com/users/paulmck/scalability/paper/N2745r.2009.02.27a.html> [查看日期: 2009年4月5日]
，发布于2009年2月。

[19] SITES, R. L., AND WITEK, R. T. *Alpha AXP Architecture*, 第2版. 数字出版社, 1995.

【20】 SPARC INTERNATIONAL.
The SPARC Architecture Manual, 1994.